Análisis y Diseño de Sistemas II
Trabajo Práctico Nro. 1-Spring Framework
Septiembre 2025

Contenidos

1. **Diseño de servicios**: Realizar el modelo de diseño estático y dinámico con un diagrama de clases y uno de secuencia, para cada uno de los códigos de los siguientes ejercicios.
2. **Implementación de servicios**: Implementar los servicios con Spring Framework.

Fecha de entrega: 16 de Septiembre.

Actividades

- Ejercicio 1

  Implementar un servicio RESTful en la ruta `/GET /celsiusAfahrenheit/:celcius` que convierta grados Celsius a Fahrenheit.

  Request:

```
GET /celsiusAfahrenheit/100
```

  Response:

```
{
  "celsius": 100,
  "fahrenheit": 212
}
```

- Ejercicio 2

  Implementar un servicio RESTful en la ruta `GET /esPrimo/:numero` que determine si el número proporcionado es primo o no.

  Request:

```
GET /esPrimo/17
```

Response:

```
    {
     "numero": 17,
     "esPrimo": true
    }
```

- Ejercicio 3

  Implementar un servicio RESTful en la ruta `GET /invertir/:cadena` que devuelva la cadena invertida.

  Request: `GET /invertir/hola`

  Response:

```
    {
     "original": "hola",
     "invertida": "aloh"
    }
```

- Ejercicio 4

  Implementar un servicio RESTful en la ruta: `POST /estadisticas` que, dado un conjunto de números, devuelva estadísticas simples.

  - Request (JSON):

```

    {
    numeros: [10, 20, 30, 40, 50]
    }
```

  - Response:

```
{
  "promedio": 30,
  "min": 10,
  "max": 50,
  "cantidad": 5
}
```

- Ejercicio 5

  Implementar un servicio RESTful para la gestión de libros (CRUD básico en memoria). Considerar:

  - Entidad: Libro (`id, titulo, autor, anioPublicacion`)
  - Repositorio en memoria: `List<Libro>` dentro de un `@Service`.
  - Endpoints:
    * `GET /libros`: lista todos los libros
    * `POST /libros`: agrega un libro
    * `GET /libros/id`: busca por id
    * `DELETE /libros/id`: elimina un libro

- Ejercicio 6

  Implementar un servicio RESTful que sea una agenda de notas Considerar:

  - Nota (`id, titulo, contenido, fechaCreacion`)
  - Datos guardados en una lista (`ArrayList`).
  - Endpoints:
    * `GET /notas`: muestra todas las notas
    * `POST /notas`: crea una nota (el id se autogenera)
    * `PUT /notas/id`: actualiza título o contenido
    * `DELETE /notas/id`: elimina la nota

- Ejercicio 7

  Implementar un servicio RESTful que gestiona películas Considerar:

  - Pelicula (`id, titulo, director, anio, genero`)
  - Repositorio: lista en memoria.
  - Endpoints:
    * `GET /peliculas`: lista todas
    * `GET /peliculas/buscar?titulo=Matrix`: busca por título (usar `@RequestParam`)
    * `POST /peliculas`: agrega una película
    * `DELETE /peliculas/id`: elimina la película id

- Ejercicio 8

  Implementar un servicio RESTful que dado un horario en una zona, convierta a otra zona horaria. Considerar:

  - Request:

```
GET /hora?fecha=2025-08-28T20:00&origen=America/Argentina/
   Buenos_Aires&destino=Europe/Madrid
```

  - Response:

```
{
  "origen": "2025-08-28T20:00",
  "destino": "2025-08-29T01:00"
}
```

  - Librería java.time.

- Ejercicio 9

  Implementar un servicio RESTful que consuma el servicio de alguna API disponible.

- Ejercicio 10

  Implementar un servicio RESTful que consuma algún servicio disponible en:

  `https://www.postman.com/cs-demo/public-rest-apis/collection/tfzpqfc/public-rest-a`